مستندات فاز دوم پروژه
عامل هوشمند روی سامانه RAG فاز اول

پروژه بازیابی محصول ترب

۱۵ خرداد ۱۴۰۵

چکیده

هدف فاز دوم، ساخت یک لایه عامل هوشمند روی سامانه بازیابی محصول فاز اول است. این عامل از خروجی‌های فاز اول به عنوان ابزار بازیابی استفاده می‌کند، از طریق نقطه پایانی `/chat` قابل ارزیابی است، حافظه کوتاه‌مدت را بر اساس `chat_id` نگه می‌دارد، و پاسخ‌ها را در قالب مورد انتظار داور شامل `message`، `base_random_keys` و `member_random_keys` تولید می‌کند.

فهرست مطالب

۱ محل قرارگیری پروژه ۲

۲ خلاصه پیاده‌سازی بر اساس خواسته‌های مستند پروژه ۲

۳ معماری کلی ۳

۴ اجزای اصلی سامانه ۳
۱.۴ سرور HTTP . . . ۳
۲.۴ هسته عامل . . . ۳
۳.۴ ابزار RAG فاز اول . . . ۴
۴.۴ حافظه . . . ۴
۵.۴ مانیتورینگ . . . ۴

۵ قرارداد API ۵
۱.۵ نقطه پایانی . . . ۵
۲.۵ قالب درخواست . . . ۵
۳.۵ قالب پاسخ . . . ۵

۶ پیاده‌سازی سناریوهای خواسته‌شده ۵
۱.۶ سناریو اول: سوال درباره ویژگی یک محصول . . . ۵
۲.۶ سناریو دوم: سوال درباره فروشنده یا قیمت . . . ۶
۳.۶ سناریو سوم: پیدا کردن فروشنده در چند مرحله . . . ۶
۴.۶ سناریو چهارم: مقایسه دو یا چند محصول . . . ۶
۵.۶ سناریو پنجم: تصویر به محصول . . . ۶

۷ گاردریل‌ها ۶

۸ روش اجرا **۷**
۱.۸ نصب نیازمندی‌ها ۷
۲.۸ اجرای سرور ۷
۳.۸ تست در مرورگر ۷
۴.۸ بررسی سلامت ۷

۹ روش تست **۷**
۱.۹ تست‌های آماده ۷
۲.۹ دموی آماده برای ارائه ۸
۳.۹ تست دستی با PowerShell ۸

۱۰ نمونه خروجی **۸**

۱۱ محدودیت‌ها **۸**

۱۲ جمع‌بندی **۹**

۱ محل قرارگیری پروژه

کدهای فاز دوم به صورت جدا از فاز اول در مسیر زیر قرار گرفته‌اند:

```
<PROJECT_ROOT>\phase2
```

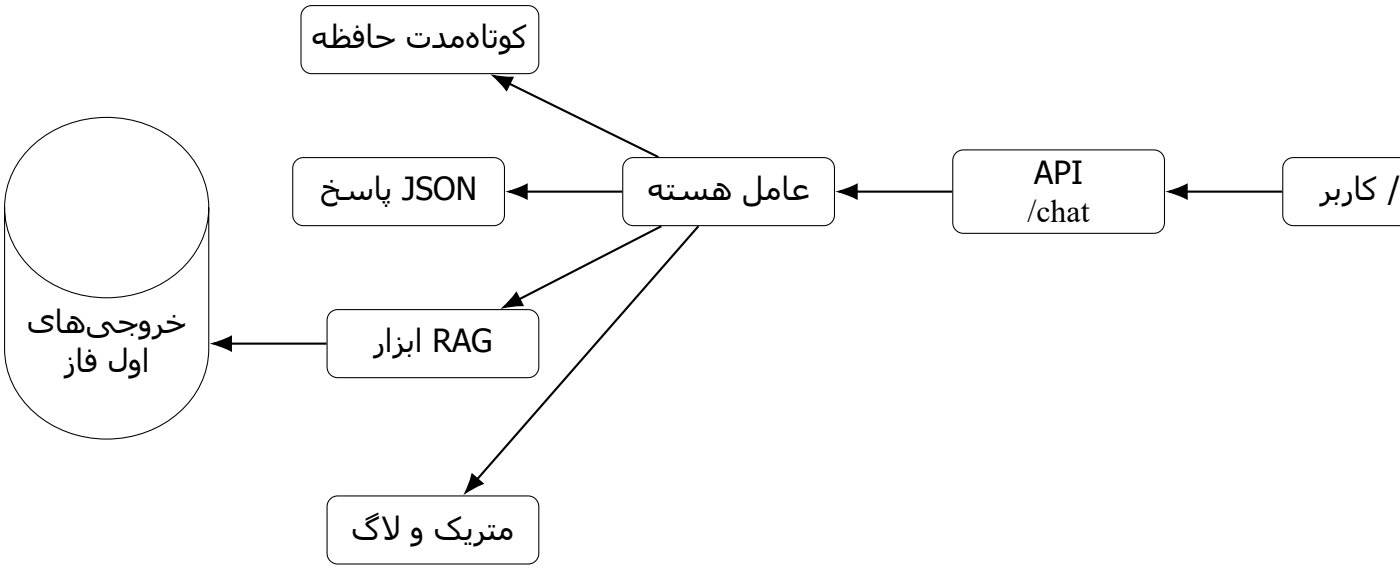
این پوشه، کدهای فاز اول را تغییر نمی‌دهد و فقط از خروجی‌های آماده فاز اول استفاده می‌کند. خروجی‌های فاز اول از مسیر زیر خوانده می‌شوند:

```
<PROJECT_ROOT>\phase1\torob_rag\artifacts
```

۲ خلاصه پیاده‌سازی بر اساس خواسته‌های مستند پروژه

بخش خواسته‌شده در مستند	کاری که انجام شده است	فایل‌های اصلی
ساخت عامل با قابلیت استفاده از ابزار	عامل نوع درخواست کاربر را تشخیص می‌دهد و هر زمان داده محصول لازم باشد ابزار RAG را فراخوانی می‌کند.	agent.py، rag.py
دسترسی به RAG فاز اول از طریق ابزار	داده‌های پاک‌سازی‌شده فاز اول به وسیله کلاس ابزار RAG خوانده و جست‌وجو می‌شوند.	rag.py
حافظه کوتاه‌مدت	پیام‌ها و اطلاعات مکالمه بر اساس chat_id ذخیره می‌شوند.	memory.py
رابط API برای داوری	نقطه پایانی POST /chat دقیقا با قالب خواسته‌شده پیاده‌سازی شده است.	server.py
رابط ساده برای تست	علاوه بر API، یک صفحه ساده در مسیر / و یک CLI نیز فراهم شده است.	server.py، cli.py
مانیتورینگ و لاگ	فراخوانی ابزار، خواندن و نوشتن حافظه، پاسخ عامل و متریک‌ها در فایل‌های محلی ذخیره می‌شوند.	monitoring.py
گاردریل‌ها	درخواست‌هایی که می‌خواهند پرامپت سیستم یا کانتکست پنهان را ببینند رد می‌شوند.	agent.py
سناریوهای تست	تست‌های دودویی و چند سناریوی نمونه برای بررسی API آماده شده‌اند.	tests/، docs/test_scenarios.md

۳ معماری کلی



شکل ۱: معماری کلی سامانه عامل هوشمند فاز دوم

۴ اجزای اصلی سامانه

۱.۴ سرور HTTP

سرور با کتابخانه استاندارد پایتون یعنی `http.server` ساخته شده است و چهار مسیر اصلی دارد:

- `GET /`: صفحه ساده برای تست دستی در مرورگر.
- `GET /health`: بررسی سلامت سرویس.
- `GET /metrics`: نمایش متریک‌های ساده اجرا.
- `POST /chat`: نقطه پایانی اصلی برای داوری.

فایل مربوط:

```
phase2/src/agentic_rag/server.py
```

۲.۴ هسته عامل

هسته عامل درخواست کاربر را دریافت می‌کند و مشخص می‌کند که باید کدام مسیر پاسخ‌دهی اجرا شود. دسته‌های تشخیص داده‌شده عبارت‌اند از:

- جست‌وجوی محصول،
- سوال درباره ویژگی یک محصول،
- سوال درباره قیمت یا فروشنده،
- پیدا کردن فروشنده در گفت‌وگوی چندمرحله‌ای،
- مقایسه چند محصول،

- درخواست تصویری،
- درخواست نامرتبط یا درخواست ناامن.

فایل مربوط:

```
phase2/src/agentic_rag/agent.py
```

۳.۴ ابزار RAG فاز اول

ابزار RAG داده‌های واقعی فاز اول را می‌خواند. فایل‌های اصلی مورد استفاده:

```
torob_rag/artifacts/clean/unified_base_products.parquet
torob_rag/artifacts/clean/members_clean.parquet
torob_rag/artifacts/embeddings/clip_v1/image/image_index.parquet
```

این ابزار سه عملیات اصلی دارد:

- `search_products`: جست‌وجوی محصول در سطح Product. Base
- `get_members`: بازیابی محصولات فروشگاهی یا Product Member برای یک .Base
- `image_to_base`: برگرداندن یک کلید Base از ایندکس تصویری فاز اول.

فایل مربوط:

```
phase2/src/agentic_rag/rag.py
```

۴.۴ حافظه

حافظه بر اساس `chat_id` کار می‌کند. اگر داور چند درخواست پشت سر هم با یک `chat_id` بفرستد، سیستم اطلاعات قبلی همان مکالمه را نگه می‌دارد.
موارد ذخیره‌شده:

- پیام‌های اخیر کاربر و عامل،
- شهر کاربر،
- بودجه،
- سبک یا جنس مدنظر،
- نیت محصولی کاربر.

حافظه هنگام اجرا در مسیر زیر ذخیره می‌شود:

```
phase2/runtime/memory.json
```

۵.۴ مانیتورینگ

برای مانیتورینگ سبک، رویدادها و متریک‌ها در فایل ذخیره می‌شوند. موارد ثبت‌شده شامل فراخوانی ابزار، خواندن و نوشتن حافظه، پاسخ عامل و خطاهای احتمالی است.

```
phase2/logs/events.jsonl
phase2/logs/metrics.json
```

متریک‌ها:

- میانگین زمان پاسخ‌دهی،
- تعداد تقریبی توکن به ازای هر پیام،
- نرخ برخورد حافظه یا Rate. Hit Memory

۵ قرارداد API

۱.۵ نقطه پایانی

```
POST /chat
Content-Type: application/json
```

۲.۵ قالب درخواست

```
{
  "chat_id": "test-1",
  "messages": [
    {
      "type": "text",
      "content": "USER_MESSAGE"
    }
  ]
}
```

۳.۵ قالب پاسخ

```
{
  "message": "ANSWER_TEXT",
  "base_random_keys": ["hhtgaa"],
  "member_random_keys": ["pjwyvz"]
}
```

۶ پیاده‌سازی سناریوهای خواسته‌شده

۱.۶ سناریو اول: سوال درباره ویژگی یک محصول

وقتی کاربر درباره ویژگی‌هایی مثل جنس، ظرفیت، عرض، رنگ یا ابعاد سوال می‌پرسد، عامل ابتدا محصول را در داده فاز اول جست‌وجو می‌کند. سپس مقدار ویژگی از ستون `extra_features` استخراج می‌شود.
مسیر اجرا:

```
Agent._feature_question()
Phase1RAGTool.answer_feature()
```

خروجی شامل پاسخ متنی و `base_random_keys` است.

۲.۶ سناریو دوم: سوال درباره فروشنده یا قیمت

اگر کاربر درباره فروشنده یا کمترین قیمت سوال کند، عامل ابتدا محصول پایه را پیدا می‌کند. سپس در جدول `members_clean.parquet` همه فروشنده‌های مربوط به همان `base_random_key` جست‌وجو می‌شوند.
برای سوال‌های مربوط به کمترین قیمت، ردیف‌های Member بر اساس قیمت مرتب می‌شوند.
مسیر اجرا:

```
Agent._seller_question()
Phase1RAGTool.get_members()
```

۳.۶ سناریو سوم: پیدا کردن فروشنده در چند مرحله

اگر درخواست اولیه مبهم باشد، عامل از کاربر سوال تکمیلی می‌پرسد. در پیام‌های بعدی، اگر `chat_id` یکسان باشد، اطلاعات قبلی مثل بودجه، شهر یا سبک مدنظر از حافظه خوانده می‌شود.
مسیر اجرا:

```
Agent._seller_hunt()
MemoryStore.update_slots()
```

۴.۶ سناریو چهارم: مقایسه دو یا چند محصول

در سوال‌های مقایسه‌ای، عامل محصولات ذکرشده را جداگانه جست‌وجو می‌کند و بر اساس معیار گفته‌شده در پرسش، یکی را انتخاب می‌کند.
مسیر اجرا:

```
Agent._compare()
Agent._choose_product()
```

۵.۶ سناریو پنجم: تصویر به محصول

در پیام تصویری، سیستم تصویر Base۶۴ را می‌پذیرد و یک `base_random_key` از ایندکس تصویری فاز اول برمی‌گرداند. برای ساده نگه داشتن پروژه، در زمان درخواست مدل سنگین CLIP یا VLM اجرا نمی‌شود.
مسیر اجرا:

```
Agent._image_response()
Phase1RAGTool.image_to_base()
```

۷ گاردریل‌ها

اگر کاربر بخواهد پرامپت سیستم، کانتکست پنهان یا دستورهای داخلی را مشاهده کند، عامل پاسخ رد می‌دهد. اگر سوال خارج از دامنه داده محصولات باشد، عامل اعلام می‌کند که نسبت به پاسخ اطمینان ندارد.
مسیر اجرا:

```
Agent._is_injection()
Agent.chat()
```

۸ روش اجرا

۱.۸ نصب نیازمندی‌ها

از ریشه پروژه اجرا کنید:

```
cd "<PROJECT_ROOT>\phase2"
pip install -r .\requirements.txt
```

۲.۸ اجرای سرور

```
$env:PHASE1_ARTIFACTS_DIR = "..\phase1\torob_rag\artifacts"
python .\run_server.py --host 127.0.0.1 --port 8000
```

خروجی مورد انتظار:

```
Phase 2 Agentic RAG is running at http://127.0.0.1:8000
Use POST /chat. Press Ctrl+C to stop.
```

۳.۸ تست در مرورگر

پس از اجرای سرور، آدرس زیر را باز کنید:

```
http://127.0.0.1:8000/
```

در این صفحه می‌توانید یک پیام وارد کنید و نتیجه فراخوانی /chat را ببینید.

۴.۸ بررسی سلامت

```
http://127.0.0.1:8000/health
```

پاسخ مورد انتظار:

```
{"status": "ok"}
```

۹ روش تست

۱.۹ تست‌های آماده

```
$env:PHASE1_ARTIFACTS_DIR = "..\phase1\torob_rag\artifacts"
python .\tests\smoke_test.py
python .\tests\http_smoke_test.py
```

۲.۹ دموی آماده برای ارائه

اسکریپت دمو یک سرور موقت /chat اجرا می‌کند، سناریوهای اصلی ارزیابی را به شکل درخواست HTTP واقعی می‌فرستد، فیلدهای مورد انتظار پاسخ را اعتبارسنجی می‌کند، re-quest و response هر مرحله را چاپ می‌کند، و در پایان سرور را می‌بندد.

```
python .\tests\presentation_demo.py
```

۳.۹ تست دستی با PowerShell

```
$body = @{
  chat_id = "test-1"
  messages = @(
    @{
      type = "text"
      content = ""□□□□□□□" □□□□ □□□□ □□□ □□□□□□□□ □□□□□□□ □ □□□□□□□□ □□ 1375
   □□□□ □□□□"
    }
  )
} | ConvertTo-Json -Depth 5

Invoke-RestMethod `
  -Method Post `
  -Uri "http://127.0.0.1:8000/chat" `
  -ContentType "application/json; charset=utf-8" `
  -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
```

۱۰ نمونه خروجی

پرسش نمونه:

کمترین قیمت برای ماگ هندوانه فانتزی و کارتونی کد ۱۳۷۵ چقدر است؟

خروجی نمونه:

```
{
  "message": "ANSWER_TEXT_IN_PERSIAN",
  "base_random_keys": ["hhtgaa"],
  "member_random_keys": ["pjwyvz"]
}
```

در اجرای واقعی، متن پیام شامل توضیح فارسی قیمت است و مقدار قیمت از جدول members_clean.parquet استخراج می‌شود.

۱۱ محدودیت‌ها

- بازیابی متنی به صورت جست‌وجوی واژگانی ساده روی داده پاک‌سازی‌شده فاز اول انجام می‌شود تا پروژه روی سیستم فعلی به راحتی اجرا شود.

- برای تصویر، سیستم از ایندکس تصویری فاز اول یک کلید Base برمی‌گرداند و در لحظه درخواست مدل سنگین بینایی اجرا نمی‌کند.
- پاسخ‌دهی بدون API خارجی LLM انجام می‌شود و بر اساس شواهد بازیابی‌شده قطعی و قابل تکرار است.
- در اولین اجرای سرور، بارگذاری فایل‌های Parquet فاز اول ممکن است کمی زمان ببرد.

۱۲ جمع‌بندی

این پیاده‌سازی نیازهای اصلی فاز دوم را با ساختاری ساده و قابل اجرا پوشش می‌دهد. عامل روی داده واقعی فاز اول کار می‌کند، API مورد نیاز داور را فراهم می‌کند، حافظه کوتاه‌مدت دارد، برای پاسخ‌ها کلیدهای Base و Member را در قالب درست برمی‌گرداند و لاگ و متریک پایه نیز تولید می‌کند.